

Analog Anchor: iOS Architecture Guide

Building Reboot-Proof Offline Challenges & Focus Modes on Apple iOS (No VPN Needed)

1. The Mindset Shift: Android vs. iOS

On Android, an offline challenge app acts like a security guard constantly running in the background, managing dummy VPN routing and listening to boot/unlock broadcasts. On iOS, third-party apps cannot sit in the background or monitor other apps. Instead, iOS utilizes an **Orchestrator Model**:

Key Principle: Your iOS app acts as a Controller that registers rules with Apple's operating system kernel (Screen Time subsystem). Once registered, iOS itself enforces the blocks, renders the unskippable shields, and manages the timers across reboots—even if the app is terminated.

2. The 4 Pillars of Apple's Screen Time Suite

1. FamilyControls (Authorization & Privacy-Safe Selection)

- **Authorization:** App requests FaceID/TouchID permission via `AuthorizationCenter.shared.requestAuthorization(for: .individual)`.
- **Privacy Picker:** Apple presents `FamilyActivityPicker`. The user selects target apps or categories (e.g. All Social, All Entertainment, All Web Browsers).
- **Security:** The app receives opaque tokens (`FamilyActivitySelection`) rather than raw package names.

2. ManagedSettings (The System Shield & Anti-Uninstall Engine)

- **Block Apps:** `store.shield.applications = selection.applicationTokens`
- **Block All Web Traffic:** `store.shield.webDomainCategories = .all()` (completely shuts down Safari & WebKit internet browsing).
- **Anti-Uninstall Lock:** `store.application.denyAppRemoval = true` (iOS physically removes the 'Delete App' button from the home screen while active!).

3. ShieldConfiguration & ShieldAction (Custom UI & Zero Bypass)

- **System Shield:** When a blocked app is opened, iOS replaces the screen with a full-screen system overlay.
- **Custom Branding:** `ShieldConfigurationExtension` provides custom title ('Analog Anchor Active'), body copy, icon, and colors.
- **Zero Bypass:** By disabling the dismiss action in `ShieldActionExtension`, the user has zero bypass options.

4. DeviceActivity (Reboot-Proof Timer Engine)

- **Native Scheduling:** App registers a `DeviceActivitySchedule` (e.g. 2 hours) with `DeviceActivityCenter`.
- **Survives Reboots:** Timer is tracked inside the iOS kernel. If the iPhone restarts, iOS verifies the active schedule on boot and keeps the phone locked.
- **Automatic Unlock:** When time expires, iOS wakes `DeviceActivityMonitorExtension` to clear the store and unlock.

3. Why the 'No VPN' Approach is Superior on iOS

Feature / Metric	Legacy VPN Approach	Native Screen Time Approach
Battery Consumption	Drains battery routing dummy packets	0% extra battery (handled at OS level)
Bypass Vulnerability	User can toggle off in Settings > VPN	Unskippable Shield + denyAppRemoval
Reboot Handling	Requires reconnect loops / boot tricks	Native OS persistence across reboots

App Store Review	High scrutiny under VPN Guideline 5.4	Officially endorsed for digital wellbeing
User Experience	Persistent VPN icon in status bar	Clean, polished Apple-native UI

4. Recommended Xcode Project Structure

```

AnalogAnchor-iOS/
├── AnalogAnchorApp/                                # Main iOS App Target (SwiftUI)
│   ├── App/ (AnalogAnchorApp.swift)
│   ├── Views/ (HomeView.swift, ChallengeActiveView.swift, AppPickerSheet.swift)
│   ├── Services/ (ScreenTimeManager.swift - Coordinates shields & schedules)
│   └── Shared/ (App Group for shared state via UserDefaults)
├──
├── DeviceActivityMonitor/                          # Extension 1: Background Timer Engine
│   └── DeviceActivityMonitorExtension.swift (Handles interval start/end)
├──
├── ShieldConfiguration/                          # Extension 2: Custom Shield UI
│   └── ShieldConfigurationExtension.swift (Custom text, icons, colors)
├──
└── ShieldAction/                                  # Extension 3: Shield Button Handler
    └── ShieldActionExtension.swift (Handles user taps on the shield)

```

5. Prerequisites for iOS Development

- 1. **Mac computer** running macOS with **Xcode** installed.
- 2. **Apple Developer Program Account** (Individual or Organization).
- 3. **FamilyControls Capability** enabled in your Apple Developer Portal.
- 4. **App Group** configured in Signing & Capabilities to share timer data across targets.